Project Workflow

EduGenie - AI Learning Assistant

1. Introduction

This document explains how EduGenie works from the moment a student opens the page until the answer is shown, how the work was organised into development stages, and why the workflow is designed this way. The whole system is built around one idea: **one simple page, many specialised AI tools behind it**.

2. End-to-End Request Workflow

1. Open the portal	The browser sends GET /. Main.py returns index.html with response=None, an empty prompt and task="concept", so the page opens clean with Concept Explanation selected.
2. Choose a task	The student picks one of five options in the drop-down. Each option shows which model will answer, for example "Concept Explanation (Local LaMini-Flan-T5)".
3. Enter the query	The student types a question, topic or text into the textarea (marked required, so the browser blocks an empty submission).
4. Submit	The form posts prompt and task to /generate.
5. Validate	FastAPI converts task into the TaskType enum. Any value outside concept, qa, summary, quiz or path is rejected automatically.
6. Route	An if/elif chain in generate_response() calls the matching module and sets the model label.
7. Generate	Concept: local LaMini model. Other tasks: a task-specific prompt is built and sent to Gemini through generate_with_retry().
8. Handle errors	Any exception is caught; the page shows "Error generating response: ..." with the label "System Error".
9. Display	index.html is rendered again with the answer, the original prompt, the chosen task and a badge showing the model used.

3. Task Routing

Task value	Function called	Model	Typical output
concept	explanation_module.get_concept_explanation	LaMini-Flan-T5-783M (local CPU)	Short explanation (up to 256 new tokens)
qa	qna.get_answer	Gemini (Google Cloud)	Detailed answer with practical examples
summary	summary_module.summarize_text	Gemini (Google Cloud)	1-2 sentence overview and bullet takeaways
quiz	quiz_module.generate_quiz	Gemini (Google Cloud)	3 multiple-choice questions and an answer key

Task value	Function called	Model	Typical output
path	learning_path.generate_roadmap	Gemini (Google Cloud)	Phased roadmap with topics, projects, timelines

4. Start-up Workflow

- **Load configuration:** python-dotenv reads the .env file next to Main.py.
- **Check the key:** the API key must exist and must not be an OAuth token; otherwise start-up stops with a clear message.
- **Create one Gemini client:** a single client is created once and passed into every Gemini-based function.
- **Mount resources:** /static is served for style.css and templates/ is registered for Jinja2.
- **Keep the local model idle:** the LaMini model is not loaded yet. It loads on the first Concept request, so the server starts quickly and uses no model memory unless that feature is used.

5. Development Workflow

The project was built in three stages, called epics. Each one produces something the next one depends on, so the order matters.

Stage	Question answered	Result	Used by
Model Selection and Architecture	Which models, and how should the code be organised?	Hybrid local + Gemini design, one module per task, layered structure	Core Functionalities
Core Functionalities Development	How does each learning tool actually work?	Five task modules, prompts, validation, routes, error handling	Frontend
Frontend Development	How does the student use all this?	index.html form, style.css, answer box with model badge	Testing and Deployment

6. Why this Workflow is Useful

- **Clear separation:** the page knows nothing about models; the modules know nothing about HTML. Either side can be improved on its own.
- **Predictable behaviour:** every task follows the same path, so errors are easy to trace.
- **Cost control:** the free local model handles the simplest task, and the paid cloud model is used only where it adds value.
- **Easy extension:** a new learning tool needs one new module, one enum value, one routing branch and one drop-down option.